

ジャガイモ

- ・ 制限時間: 10 分
- ・ 環境: GPU 1 基 (≈16 GB VRAM) 、インターネットなし
- ・ 提出物のサイズ: solution.ipynb ≤ 1 MB
- ・ ストレージ: 5 GB

課題

あなたの友人が推測ゲームをしようと提案します。ジャッジ役である友人は、固定された語彙から単語を 1 つ選び、あなたは 30 ターン以内にそれを見つけてなければなりません。各ターンで、ジャッジは 2 つの単語を比較し、どちらが単語に意味的に近いかを報告します。すべてのゲームは固定された組 lamp vs potato から始まります。これは、それらがあなたの友人の特に好きなもの 2 つだからです。その後、あなたのプログラムは新しい単語を 1 つ提案します。比較で勝った単語は保持され、次の提案と比較されます。単語を正確に提案した時点で、ゲームに勝利します。一致判定では大文字と小文字を区別しません。提案するすべての単語は dataset/vocabulary.json に含まれていなければなりません。

プロトコルとデータの読み込みを含む完全な例が solution.ipynb にあります。PublicEmbeddingPlayer クラスは実装できます。プログラムは 1 度だけ初期化され、1 回の実行ですべてのゲームをプレイします。プロトコルは各ゲームの開始時に新しい PublicEmbeddingPlayer を作成します。

ジャッジ

あなたのプログラムはジャッジに JSON オブジェクトを 1 つ送信し、ジャッジは JSON オブジェクトを 1 つ返します。

プロトコルを説明するただけに単語を示した例は、次のとおりです。

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp",   "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock",   "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock",   "word2": "hammer"}
-> {"new_word": "shovel"}
                                     <- matches: game won
-> {"status": "win"}
```

ターンには 1 から 30 までの番号が付けられます。

verdict の選択肢は、word1 のほうが近いことを意味する first、word2 のほうが近いことを意味する second、または単語の単語が単語に同じ程度に近いことを意味する same です。

winner_word は、次の比較のために保持される単語です。same という判定の場合、最初の単語が残ります。

データセット

すべての split で共有されるものは次のとおりです。

- ・ dataset/vocabulary.json — 1602 個の一意な小文字の単語の単語は常にこれらのうちの 1 つです。
- ・ dataset/public_embeddings.npy — float32 shape は (1602, 2560) * 行 i は、語彙内の単語 i に属します。これらは公開 embedding です。ジャッジは単語の非公開の表現(embedding)を使用します。

各 split は単語の集合です。

Split	単語	正解	用途
test_public	120	dataset/test_public.json	solution を実行し、自己点する
test_leaderboard_a	120	非公開	ライブラリテストボード
test_leaderboard_b	120	非公開	最終順位

train split はありません。ラベル付きの行から fitting されるものは何也没有ません。

提供されるモデル

事前学習済み **embedding** モデルが 2 つ、この課題に同梱されており、使用できます。

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

どちらもローカルパスから読み取らなければなりません。"**BAAI/bge-m3**" のような **Hugging Face hub id** を使用するとダウンロードが失敗し、ジャッジ環境はオフラインであるため失敗します。各ディレクトリには、オフラインでの呼び出しを示すような `example.py` が含まれています。

利用可能なライブラリは `numpy`、`torch`、`sentence-transformers` です。インタプリタ、ダウンロード、その他のパッケージは利用できません。

出力

ありません。これは空白です。**solution** は解答ファイルを書き出さず、上記のとおり **stdin/stdout** を介してジャッジと通信します。

評価指標

タスクで出したゲムのスコアは $1.0 - 0.02 \times \max(0, t - 10)$ です。30 タク以内には解けなかったゲムのスコアは 0 です。したがって、タスク 1-10 のスコアは 1.00、タスク 20 のスコアは 0.80、タスク 30 のスコアは 0.60 です。

課題のスコアはゲムスコアの平均 $\times 100$ であり、0.00 から 100.00 までです。

10 分の制限時間は、起動、準備、および **test set** の全 120 ゲムを含む時間の合計です。

提出方法

1. `solution.ipynb` を開き、`PublicEmbeddingPlayer` を編集して、正しく動作することを確認するためにすべてのセルを実行します。
2. 任意で、ローカルで確認します: `python local_test.py solution.ipynb --limit 5`。ローカルジャッジは公開 **embedding** を使用するため、そのスコアは参考値にすぎません。
3. `solution.ipynb` を保存します。
4. JupyterLab の左サイドバーにある **Git** タブを開きます。
5. `solution.ipynb` をステージします（その横にある **+** アイコン）。
6. **commit message** を入力し、**Commit** をクリックします。
7. 上向き矢印付きのクラウドをクリックして **push** します。
8. このコンテストページにあり、入力したものと一致する **commit message** を指定して **Submit** をクリックします。

必要な準備と **inference** のすべてを含む、`solution.ipynb` という名前のファイルだけを提出してください。